

Arquitectura de Computadoras II - Resumen

CLASE 1

Circuitos Lógicos y Conceptos Básicos

Los sistemas están constituidos por circuitos lógicos. El matemático Boole escribió un álgebra sobre el pensamiento basado en variables lógicas, las cuales tienen dos valores posibles: verdadero o falso. Esto da lugar a las variables booleanas, que solo admiten dos valores (0 y 1).

[ESPACIO PARA IMAGEN: ESQUEMA DE PUERTAS LÓGICAS]

CLASE 4

Memoria RAM

Es posible construir una memoria utilizando *flip-flops* de tipo D-Latch. Estos funcionan con una entrada de datos (D), una señal de reloj (Clock) y una salida (Q).

Si unimos 4 *flip-flops* D-Latch conectando sus relojes con una línea común:

- Si el reloj está en **0**, están reteniendo la información.
- Si el reloj está en **1**, están copiando la entrada.

Para seleccionar qué parte queremos que se copie, se utiliza un **decodificador**. Este dispositivo tiene n entradas y 2^n salidas. Para cada combinación de datos de entrada, una única salida se pone en 1.

- Con una entrada 01 en el decodificador, se activaría esa línea específica y se copiaría el número en esa parte de la memoria.

Lectura y Escritura

La memoria RAM posee una línea de control de lectura y escritura (L/E).

- Cuando hay un **1**, lee.
- Cuando hay un **0**, escribe.

Para implementar esto, se coloca una compuerta AND delante de los primeros componentes. A la línea de lectura se le añade un inversor conectado a esta compuerta, y la otra entrada se conecta a la salida del decodificador.

Para leer, se puede utilizar un mecanismo parecido al múltiple XOR (usando compuertas AND), donde la salida del decodificador conecta con una línea que llega a todas las AND.

[ESPACIO PARA IMAGEN: ESQUEMA INTERNO DE MEMORIA RAM Y DECODIFICADOR]

El dato final sale y va al procesador. Se colocan 4 compuertas *tri-state* conectadas a la misma línea de control. Cuando se escribe (0), la señal llega a esta compuerta y los *tri-state* se desactivan para no interferir.

Memoria ROM

Se encuentra dentro de la unidad de control. En este tipo de memoria los datos están fijos (se fabrican con la información dentro). La información queda firme desde el momento de su fabricación y no se puede escribir, solo leer. Del esquema anterior, todo lo relacionado con la escritura se descarta, conservando solo la lógica de lectura.

Registros

Gracias al flip-flop D-Latch pudimos armar la memoria; de igual manera, se pueden armar registros.

En un registro simple, un único contacto de reloj se une a todos los flip-flops, con las entradas D arriba y las salidas Q abajo. El dato siempre está guardado en Q.

Sin embargo, este circuito es inestable en ciertas configuraciones (como en un bucle con la UAL), ya que se repite todo el tiempo. Si la UAL (Unidad Aritmético Lógica) recibe un 1 en el reloj, el *latch* establece una conexión directa entre la salida y la entrada de la UAL. La UAL no admite que la salida se inyecte en la entrada sin control.

Registros con Flip-Flop Maestro-Esclavo

Para solucionar la inestabilidad, se reemplaza con una configuración **Maestro-Esclavo**.

- El reloj se invierte (se niega) para una de las partes.
- Primero copia el maestro mientras el esclavo retiene.
- Lo que importa es lo que retiene el **esclavo**, porque es el dato que podemos leer a la salida del registro.

- Cuando el reloj pasa a 0, se retiene la respuesta, pero al pasar a 1, el valor que quería entrar entra al maestro.

Lo que evita el bucle infinito es que, cuando llega el nuevo valor, se encuentra con maestros que ya están reteniendo un valor anterior, cerrando el paso al nuevo dato hasta el siguiente ciclo.

Datapath

El *Datapath* organiza cómo el procesador realiza el intercambio de información interna, relacionando la salida de los registros y las entradas con la UAL. La UAL permite mover información desde la salida del registro a la entrada.

[ESPACIO PARA IMAGEN: ORGANIZACIÓN DEL DATAPATH]

- La salida de los registros pasa por compuertas *tri-state* con su línea de control (uno para cada bus).
- La entrada de los registros está conectada a los *flip-flops* maestro-esclavo.
- Para controlar el reloj de los registros, se usa una compuerta AND conectada al reloj global y a una línea de control que viene de la Unidad de Control (UC).

Componentes del Datapath:

- **Registros:** Con acceso a los buses y a la UAL.
- **Bus X y Bus Y:** Transportan datos a las entradas de la UAL.
- **Bus W:** Transporta el resultado de la UAL de vuelta a los registros.
- **Registro Auxiliar:** Vale siempre 0. Sirve para usar la UAL como "conector" (sumando 0 al valor para transportarlo sin modificarlo).

CLASE 5

Instrucciones y Modos de Direccionamiento

Para generar un procesador, combinamos lógica combinacional (UAL), circuitos biestables (Flip-Flops/Registros) y memorias. Faltan las instrucciones.

Es necesario conocer la arquitectura (registros disponibles, bits, función, relación con el bus de direcciones) para entender el **Set de Instrucciones**. Las instrucciones ocupan posiciones de memoria y constan de:

1. **Código de operación:** Qué tiene que hacer.

2. **Dato asociado:** Con qué lo tiene que hacer.

Lenguaje Mnemónico (Assembler)

En lugar de usar código máquina (ej. A1 00 20), se usan mnemónicos.

Ejemplos:

- **MOV AX, [2000]:** Mueve el contenido de la dirección 2000 al registro AX.
- **ADD AX, [2002]:** Suma el contenido de la dirección 2002 a AX y guarda el resultado en AX.
- **SUB AX, [2004]:** Resta el contenido de la dirección 2004 a AX.
- **MOV [2006], AX:** Guarda el valor de AX en la dirección 2006.

El software que convierte estos mnemónicos a lenguaje máquina se llama **Ensamblador** (*Assembler*).

Modos de Direccionamiento

1. Direccionamiento Directo:

El dato asociado de la instrucción es la dirección de memoria donde se encuentra el dato a operar.

- Ejemplo: MOV AX, [2000]

2. Direccionamiento Inmediato:

El dato asociado de la instrucción es el dato mismo a operar.

- Ejemplo: SUB AX, 2004 (al no tener corchetes, 2004 es el valor numérico, no una dirección).

3. Direccionamiento Modo Registro:

El dato asociado es una referencia al registro donde se encuentra el dato.

- Ejemplo: MOV BX, AX (Copia lo de AX a BX). No hay dato asociado externo, todo es interno.

4. Direccionamiento Indexado (Indirecto):

El dato asociado es una referencia a un registro que contiene la dirección del dato a operar. Se usa para estructuras de datos variables.

- Ejemplo: MOV AX, [BX] (BX actúa como registro índice o puntero).

5. Direccionamiento Relativo:

El dato asociado es un valor que se suma al IP (Instruction Pointer) para producir un salto en la ejecución.

- Se usa en instrucciones como JMP (Jump).
- Cálculo: Nuevo IP = IP actual + Dato Asociado.

[ESPACIO PARA IMAGEN: EJEMPLO DE CÓDIGO EN ASSEMBLER O DEBUG]

CLASE 6

Ejecución de un Programa en el Modelo de Procesador

Pasos generales para ejecutar instrucciones en el modelo (Ciclo de Instrucción):

1. Fase de Pedido (Fetch):

- El **IP** apunta a la dirección de memoria de la próxima instrucción.
- Se carga esa dirección en el registro de direcciones (RDI).
- La memoria lee el dato (la instrucción) y lo coloca en el Bus de Datos.
- El dato sube y se carga en el **Registro de Instrucciones (RI)**.
- El IP se incrementa para apuntar a la siguiente instrucción.

2. Fase de Ejecución (Decode & Execute):

- La Unidad de Control (UC) decodifica lo que hay en el RI.
- Activa las líneas de control necesarias (Habilitar buses, activar UAL, señales de lectura/escritura).
- Se realizan las operaciones (movimientos de datos, sumas, restas) a través de los buses y la UAL.
- Se actualizan los *flags* (Signo, Zero, Overflow, Carry) en el registro de estado si corresponde.

Nota: El proceso se repite ciclo tras ciclo. El Registro Latch (RL) y las señales de reloj coordinan los tiempos de retención y copia (Maestro-Esclavo).

[ESPACIO PARA IMAGEN: DIAGRAMA DE TIEMPOS O CICLOS DE RELOJ]

CLASE 9

Programación en Assembler

Se utiliza el programa DEBUG (dentro de un emulador como DOSBox) para escribir y ejecutar instrucciones.

- DOSBox emula un procesador 8086 (Bus de direcciones de 20 líneas = 1 MB de memoria).
- Comandos básicos de Debug:
 - A (Assemble): Escribir código.
 - T (Trace): Ejecutar paso a paso.
 - U (Unassemble): Ver el código máquina.
 - R (Register): Ver el estado de los registros.

Registros Principales

Los registros son de 16 bits (AX, BX, CX, DX), pero se pueden dividir en dos de 8 bits (High y Low):

- $AX \rightarrow AH$ (Alto) y AL (Bajo).
- Ejemplo: `MOV AL, [1234]` mueve solo 8 bits.

Ejemplo: Multiplicación mediante sumas sucesivas

Fórmula: $R = M \times N$.

Lógica del programa:

1. Inicializar un acumulador en 0 (`SUB AX, AX`).
2. Usar un registro (ej. CX) como contador con el valor de N .
3. **Bucle:**
 - Sumar M al acumulador (`ADD AX, [Dirección de M]`).
 - Decrementar el contador (`DEC CX`).
 - Si no es cero, saltar al inicio del bucle (`JNZ`).

Manejo de errores: Se pueden usar saltos condicionales basados en *flags* para detectar errores:

- JC / JNC: Jump if Carry / No Carry (para naturales).
 - JO / JNO: Jump if Overflow / No Overflow (para enteros).
-

CLASE 11

Segmentación de Memoria y Subrutinas

Segmentación

La dirección física no es absoluta, sino una combinación de un **Segmento** y un **Desplazamiento** (Offset).

- Segmentos: CS (Code), DS (Data), SS (Stack), ES (Extra).
- Cálculo de dirección física: Se toma el número de segmento, se desplaza un lugar (hexadecimal) a la izquierda (se agrega un 0) y se le suma la dirección del *offset*.
- Esto permite tener segmentos para distintos propósitos (código, datos) que pueden estar en lugares distintos o superpuestos.

Llamado a Subrutinas (CALL y RET)

Permite reutilizar código.

1. Instrucción CALL dirección:

- El procesador guarda el valor actual del IP en la **Pila** (Stack) para saber dónde volver.
- Decrementa el **SP** (Stack Pointer) en 2 ($SP \leftarrow SP - 2$).
- Escribe el IP en la posición apuntada por SP ($[SP] \leftarrow IP$).
- Modifica el IP con la dirección de salto (Direccionamiento relativo: $IP \leftarrow IP + \text{Desplazamiento}$).

2. Instrucción RET:

- Lee el valor almacenado en la pila (donde apunta SP).
- Carga ese valor en el IP.

- Incrementa el SP en 2 ($SP \leftarrow SP + 2$).
- El programa continúa en la instrucción siguiente al CALL.

[ESPACIO PARA IMAGEN: ESQUEMA DE PILA Y CALL/RET]

CLASE 12

Interrupciones

Interrupciones por Software (INT)

Instrucción INT XX. Permite saltar a rutinas del Sistema Operativo o BIOS.

- No se especifica la dirección de salto, sino un número de vector.
- El procesador consulta la **Tabla de Vectores de Interrupción** (ubicada en 0000:0000).
- Cada vector contiene el nuevo IP y CS.
- **Proceso:**
 1. Resguarda el Registro de Estado (Flags), CS e IP en la pila.
 2. Busca el vector correspondiente ($XX * 4$).
 3. Deshabilita interrupciones (Flag I = 0).
 4. Salta a la rutina de interrupción.
 5. Vuelve con la instrucción IRET.

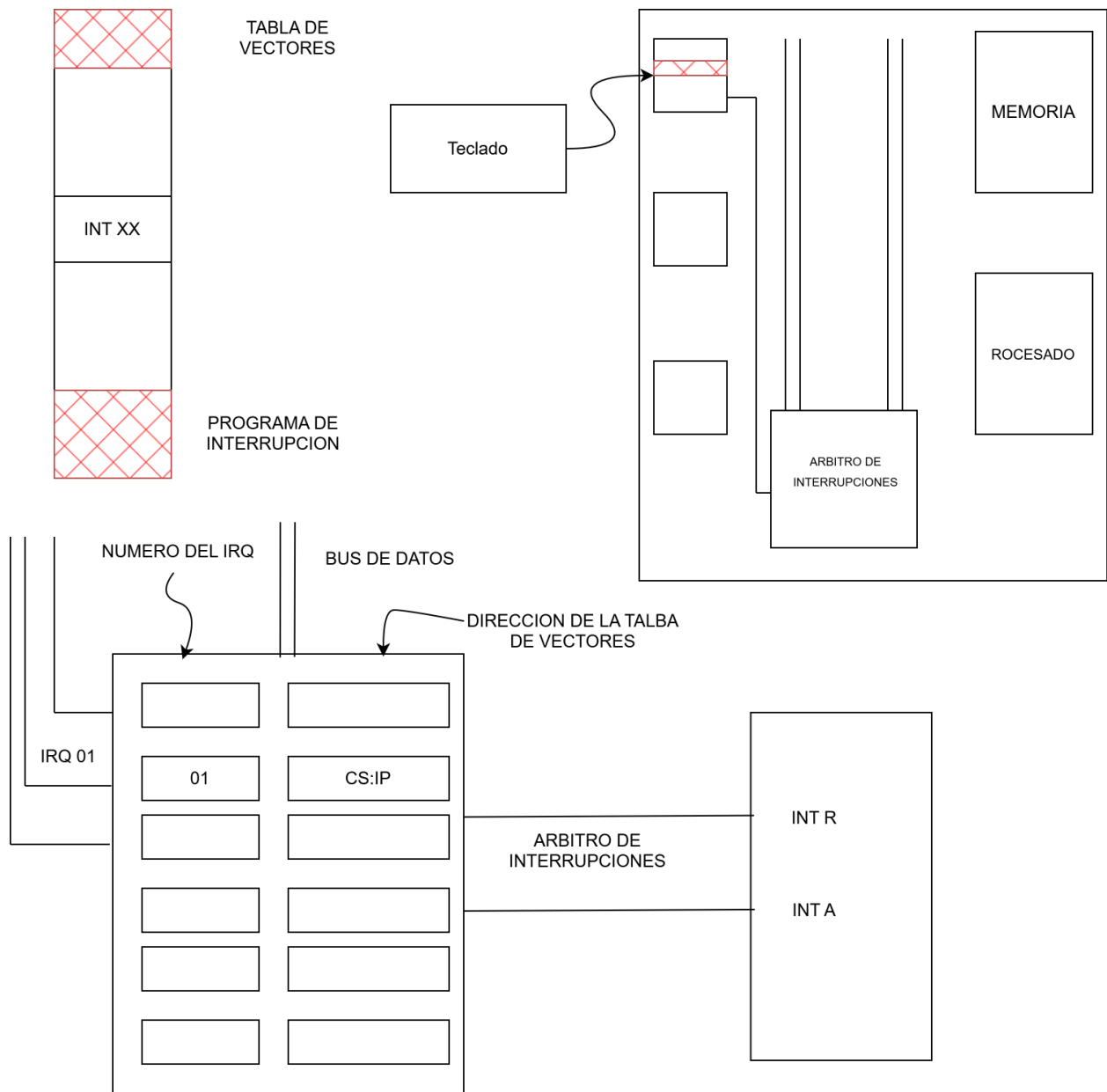
Interrupciones por Hardware

Se activan mediante señales externas en las patillas **INT R** (Interrupt Request) e **INT A** (Interrupt Acknowledge) del procesador. Existe un **Árbitro de Interrupciones** (PIC) que gestiona los periféricos. En este árbitro se guarda la configuración que relaciona el número de IRQ con el vector de interrupción correspondiente, lo cual es establecido por el programa de arranque o el Sistema Operativo.

Secuencia de una Interrupción por Hardware (Ejemplo: Teclado)

1. Al interactuar con el hardware (ej. presionar una tecla), se generan los datos correspondientes (en el teclado son 2 caracteres: uno al apretar y otro al soltar).

2. Estos datos se almacenan en el dispositivo de entrada.
3. El dispositivo de entrada envía una señal (un '1') al **Árbitro de Interrupciones**.
4. La señal llega a la línea **IRQ** correspondiente (para el teclado es la IRQ 01).
 - *Nota de Prioridad:* El número de IRQ indica la jerarquía; cuanto menor es el número, mayor es la prioridad (ej. si llega una IRQ 0, el procesador la atenderá antes que la 01).
5. El árbitro envía un '1' al pin **INT R** del procesador.
6. El procesador chequea esta línea cuando el Registro Latch (RL) cambia a 0000 (antes de pasar a la siguiente instrucción). Si el **Flag I** (Interrupción) está habilitado (**EI**), detecta el '1' en INT R.
7. Al detectar la petición, el procesador comienza a ejecutar el microcódigo para gestionar la interrupción por hardware y responde enviando un '1' por el pin **INT A**.
8. El RL toma el valor correspondiente para preparar el salto por hardware.
9. Se realiza el **resguardo en la Pila** (se guardan Flags, CS e IP), exactamente igual que en las interrupciones por software.
10. El procesador envía un segundo '1' por el pin **INT A** confirmando que está listo para el salto. En respuesta, el árbitro envía la **dirección del vector de interrupción** específica a través del Bus de Datos hacia el procesador.
11. El procesador realiza el salto a la dirección de memoria donde se encuentra el programa de atención al IRQ (Driver). Esta rutina finaliza con la instrucción IRET, permitiendo volver al punto exacto donde se había quedado el programa principal.



Instrucciones PUSH y POP

Sirven para guardar y recuperar registros en la pila manualmente. Son útiles en subrutinas para no perder los valores de los registros del programa principal.

CLASE 13

Memoria Caché

La memoria RAM principal (DRAM) es mucho más lenta que el procesador. Para mitigar esta diferencia de velocidad, se utiliza la **Memoria Caché** (SRAM).

- Es más rápida, pero de menor capacidad.
- Se ubica entre el procesador y la memoria principal.

Funcionamiento:

- La información se transfiere en bloques (ej. 64 bytes).
- **HIT:** El dato buscado está en la caché (rápido).
- **MISS:** El dato no está, se debe buscar en la RAM (lento) y copiar el bloque a la caché.

Principios de Localidad:

- **Temporal:** Es probable volver a acceder a un dato usado recientemente.
- **Espacial:** Es probable acceder a datos cercanos a los actuales.

Tipos de organización:

- **Correspondencia Directa:** Cada bloque de RAM tiene un lugar fijo posible en la caché. Sencillo, pero con riesgo de muchos *misses* si dos datos compiten por el mismo lugar.
- **Asociativa de N vías:** Los bloques pueden guardarse en varios conjuntos posibles. Más compleja pero eficiente.

Otras Cachés:

- Caché de Disco Rígido (RAM en el disco para acelerar I/O).
 - Caché de Software (ej. navegador web), distinta a la de hardware.
-

CLASE 15

Procesadores Modernos

Pipelining (Segmentación de Instrucciones)

Técnica similar a una línea de montaje industrial. Divide la ejecución de la instrucción en etapas:

1. **Fetch:** Búsqueda.
 2. **Decode:** Decodificación.
 3. **Execute:** Ejecución.
 4. **Write-back:** Escritura.
- **Sin Pipelining:** Se ejecuta una instrucción completa antes de empezar la siguiente.
 - **Con Pipelining:** Varias instrucciones están en distintas etapas simultáneamente.
 - **Superescalar:** (Como el Pentium) Puede ejecutar más de una instrucción por ciclo de reloj gracias a tener pipelines duplicados.

CISC vs RISC

Característica	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Instrucciones	Complejas, muchas operaciones en una.	Simples, uniformes, una operación por ciclo.
Decodificación	Compleja (usa microcódigos).	Simple (hardware directo).
Registros	Pocos.	Muchos (reduce acceso a memoria).
Acceso a Memoria	Frecuente.	Reducido (se trabaja en registros).
Ejemplo	x86 (Intel/AMD clásicos).	ARM, SPARC.

Hilos (Threads) y Multiprocesamiento

- **Multiprocesamiento:** Varios procesadores físicos operando en paralelo.
 - **Threading (Software):** El Sistema Operativo alterna rápidamente entre tareas (cambio de contexto) para simular simultaneidad.
 - **Hyper-Threading (Hardware):** Tecnología de Intel donde un núcleo físico se divide en dos **núcleos lógicos**. Comparte recursos de ejecución, pero permite gestionar dos hilos a la vez, reduciendo tiempos muertos.
-

CLASE 16

Buses y Estructuras Modernas

Actualmente, funciones como el *Northbridge* y el controlador de memoria están integrados dentro del procesador para mayor velocidad.

Bus USB (Universal Serial Bus)

- Topología en árbol. Conexión punto a multipunto.
- **Plug & Play:** Cuando se conecta un dispositivo, el controlador genera una interrupción, el SO interroga al dispositivo, carga el driver correspondiente y le asigna una dirección.

Bus PCI

- Bus paralelo (32 o 64 bits).
- Utiliza un sistema de **Maestro-Esclavo** para controlar el bus.

Tipos de Acceso a Memoria:

PIO (Periferal I/O): El método antiguo (PIO - Periferal I/O) usaba al procesador para mover datos entre disco y memoria (triangulando), lo cual era ineficiente.

Acceso Directo a Memoria (DMA): Permite que un controlador específico gestione la transferencia de grandes bloques de datos entre la memoria y los periféricos (como el disco duro) **sin intervención constante del procesador**.

1. El procesador programa al controlador DMA (origen, destino, cantidad).
2. El DMA toma el control del bus y realiza la transferencia.
3. Al finalizar, el DMA interrumpe al procesador para avisar.